



Mus1c

mus1c 



Medium

General Skills

picoCTF 2019

AUTHOR: DANNY

Description

I wrote you a [song](#). Put it in the picoCTF{} flag format.

Hints 

1

Do you think you can master
rockstar?

32,274 users solved



31%

Liked



picoCTF{FLAG}

Submit
Flag

Author: The Analyst: Hyposelenia

Challenge: Mus1c

Category: General Skills

Date: 01/18/26



I. Objective

The objective of this challenge is to analyze a text file containing song lyrics, identify the hidden programming language used, interpret its output, and decode the resulting numerical values to obtain the hidden flag.

II. Background

CTF challenges often hide information in unusual or unexpected formats to test a participant's creativity and problem-solving skills. Instead of traditional encryption, some challenges use esoteric or unconventional programming languages to disguise data.

The Muslc challenge introduces the **Rockstar programming language**, which resembles song lyrics rather than standard code. Recognizing and understanding such nontraditional tools is important in cybersecurity, as attackers may use creative techniques to obscure malicious logic or hidden data.

III. Tool Used

- **Oracle VirtualBox (Kali Linux)** – Used as the Linux environment
- **Linux Terminal** – Used to execute decoding commands
- **Rockstar Online Interpreter** – Used to execute the Rockstar code
- **Online Decimal to ASCII Converter** – Used to convert Decimal to ASCII

IV. Methodology

1. Downloaded the provided file and opened the text file containing the lyrics.
2. Copied the entire contents of the text file.
3. Searched for “Rockstar programming language” and opened an online interpreter (<https://codewithrockstar.com/>).
4. Pasted the lyrics into the Rockstar interpreter and executed the code.



```
36 shout my lyric
37 shout Pico
38 shout It
39
40 Pico CTF is fun
41 security is important
42 Fun is fun
43 Put security with fun into Pico CTF
44 Build Fun up
45 shout fun times Pico CTF
46 put fun times Pico CTF into my song
47
48 build it up
49
50 shout it
51 shout it
52
53 build it up, up
54 shout it
55 shout Pico
```

Rock ▶ 114
Reset ↺ 114
Parse 📄 111
109
110
114
110
48
49
49
51
114

Try it ↗

5.

6. Observed that the output consisted of a sequence of numbers.
7. Identified the numbers as **decimal values** representing ASCII characters.
8. Copied the decimal output and used an online decimal-to-ASCII decoder.
9. Placed the decoded text inside the format picoCTF{ }.
10. Submitted the flag.

V. Result

picoCTF{rrrocknrn0113r}



Space

☐ 0x/0b prefix

ASCII text

`rrrocknrn0113r`

Hex (bytes)

`72 72 72 6F 63 6B 6E 72 6E 30 31 31 33 72`

Binary (bytes)

`01110010 01110010 01110010 01101111 01100011 01101011 01101110
01110010 01101110 00110000 00110001 00110001 00110011 01110010`

Decimal (bytes)

`49
51
114`

Base64

`cnJyb2NrbnJuMDExM3I=`

Length (bytes)

`14`

VI. Explanation

Well... well... wel... The flag wasn't really in the lyrics, I've also never heard of a rockstar programming before, I didn't know until I conquered this challenge lol.

I've researched for anyone interested to learn:

Rockstar is an esoteric programming language designed to make code look like song lyrics or poetry. It uses English-like phrases instead of traditional syntax, which makes it confusing at first glance.

Although Rockstar is mostly created for fun, challenges like this demonstrate how:

Code does not always look like code

Hidden logic can be disguised as plain text



Creativity is often required to solve cybersecurity problems

In this challenge, the lyrics were actually a valid Rockstar program that produced numeric output when executed.

Know that the flag was not directly hidden in the lyrics. Instead, the challenge required:

- Recognizing that the lyrics were **code**
- Finding the correct interpreter
- Understanding that the output needed further decoding

This shows that **CTF challenges are not just about brute force**, but also about curiosity, pattern recognition, and creative thinking—skills that are essential in real-world cybersecurity scenarios.

Another thing, The output from the Rockstar interpreter was a list of **decimal numbers**, each corresponding to an ASCII character.

For example:

- $114 \rightarrow r$
- $114 \rightarrow r$
- $114 \rightarrow r$

When converted from decimal to ASCII, the numbers formed the readable text used in the flag.

You can try these commands used in Linux to decode without online tools.

Decimal to ASCII

```
echo "104 121 112 111 115 101 108 101 110 105 97" | awk  
'{for(i=1;i<=NF;i++) printf "%c",$i}'
```

Hex to ASCII

```
echo "54 68 65 20 41 6E 61 6C 79 73 74" | xxd -r -p
```

ASCII to Hex

```
echo "I like donuts" | xxd -p
```



VII. Conclusion

The **Mus1c** challenge highlights the importance of thinking beyond conventional tools and approaches in cybersecurity. By identifying the Rockstar programming language and decoding its numerical output, the hidden flag was successfully recovered. This challenge reinforces that creativity, adaptability, and curiosity are just as important as technical knowledge in solving CTF problems.

— **The Analyst: Hyposelenia**